

Just xgboost

June Lemieux

2024-12-14

libraries/etc

```
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.2.0 --
```

```
## v broom      1.0.7      v recipes      1.1.0
## v dials      1.3.0      v rsample      1.2.1
## v dplyr      1.1.4      v tibble       3.2.1
## v ggplot2    3.5.1      v tidyr        1.3.1
## v infer      1.0.7      v tune         1.2.1
## v modeldata  1.4.0      v workflows    1.1.4
## v parsnip    1.2.1      v workflowsets 1.1.0
## v purrr      1.0.2      v yardstick    1.3.1
```

```
## -- Conflicts ----- tidymodels_conflicts() --
```

```
## x purrr::discard() masks scales::discard()
## x dplyr::filter()   masks stats::filter()
## x dplyr::lag()      masks stats::lag()
## x recipes::step()   masks stats::step()
## * Search for functions across packages at https://www.tidymodels.org/find/
```

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.2
```

```
##
```

```
## Attaching package: 'xgboost'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      slice
```

```
library(vip)
```

```
## Warning: package 'vip' was built under R version 4.4.2
```

```
##  
## Attaching package: 'vip'
```

```
## The following object is masked from 'package:utils':  
##  
## vi
```

```
library(ROCR)
```

```
## Warning: package 'ROCR' was built under R version 4.4.2
```

```
options(scipen = 4)
```

```
## Data Processing  
# Initial data import  
customer_data <- read.csv("~/00 Merrimack/DSE6111/Course Project/customer_data_1.csv", header = TRUE)  
  
customer_data$Attrition_Flag <- ifelse(customer_data$Attrition_Flag == "Attrited Customer", 1, 0)  
customer_data$Attrition_Flag <- factor(customer_data$Attrition_Flag)  
  
# check levels  
levels(customer_data$Attrition_Flag)
```

```
## [1] "0" "1"
```

```
# set seed  
set.seed(10)  
  
# get rid of ClientNum identifier  
customer_data <- customer_data[ -c(1) ]  
  
# create data split  
data_split <- initial_split(customer_data, strata = "Attrition_Flag", prop = 0.75)  
  
# create training and test sets  
training_set <- training(data_split)  
test_set <- testing(data_split)  
  
# row count per set  
nrow(training_set)
```

```
## [1] 7595
```

```
nrow(test_set)
```

```
## [1] 2532
```

```
# use cv folds and tune  
cv_set <- vfold_cv(training_set, v = 10)
```

```

# recipe for preprocessing
attrition_recipe_xb <-
  recipe(
    Attrition_Flag ~ .,
    data = training_set
  ) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())

# model with xgboost
xgboost_model <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune() %>%
  set_engine("xgboost") %>%
  set_mode("classification")

# create workflow
xgboost_workflow <-
  workflow() %>%
  add_recipe(attrition_recipe_xb) %>%
  add_model(xgboost_model)

##### use tune_grid to find best hyperparameter values #####

xgboost_grid <- grid_latin_hypercube(
  trees(),
  tree_depth(),
  learn_rate(),
  size = 10
)

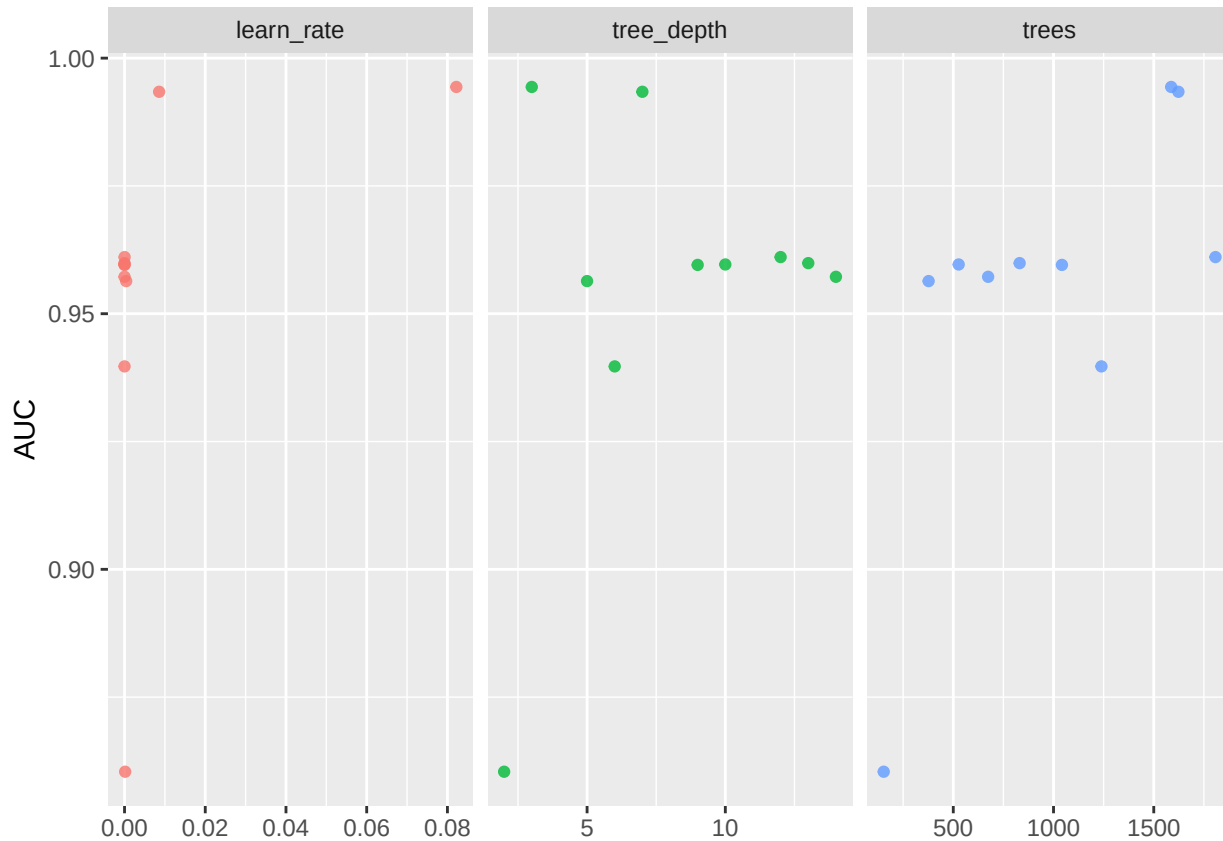
## Warning: `grid_latin_hypercube()` was deprecated in dials 1.3.0.
## i Please use `grid_space_filling()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

xgboost_results <- xgboost_workflow %>%
  tune_grid(resamples = cv_set,
    grid = xgboost_grid,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc, accuracy))

xgboost_results %>%
  collect_metrics() %>%
  filter(.metric == "roc_auc") %>%
  select(mean, trees:learn_rate) %>%
  pivot_longer(trees:learn_rate,
    values_to = "value",
    names_to = "parameter"
  ) %>%

```

```
ggplot(aes(value, mean, color = parameter)) +
  geom_point(alpha = 0.8, show.legend = FALSE) +
  facet_wrap(~parameter, scales = "free_x") +
  labs(x = NULL, y = "AUC")
```

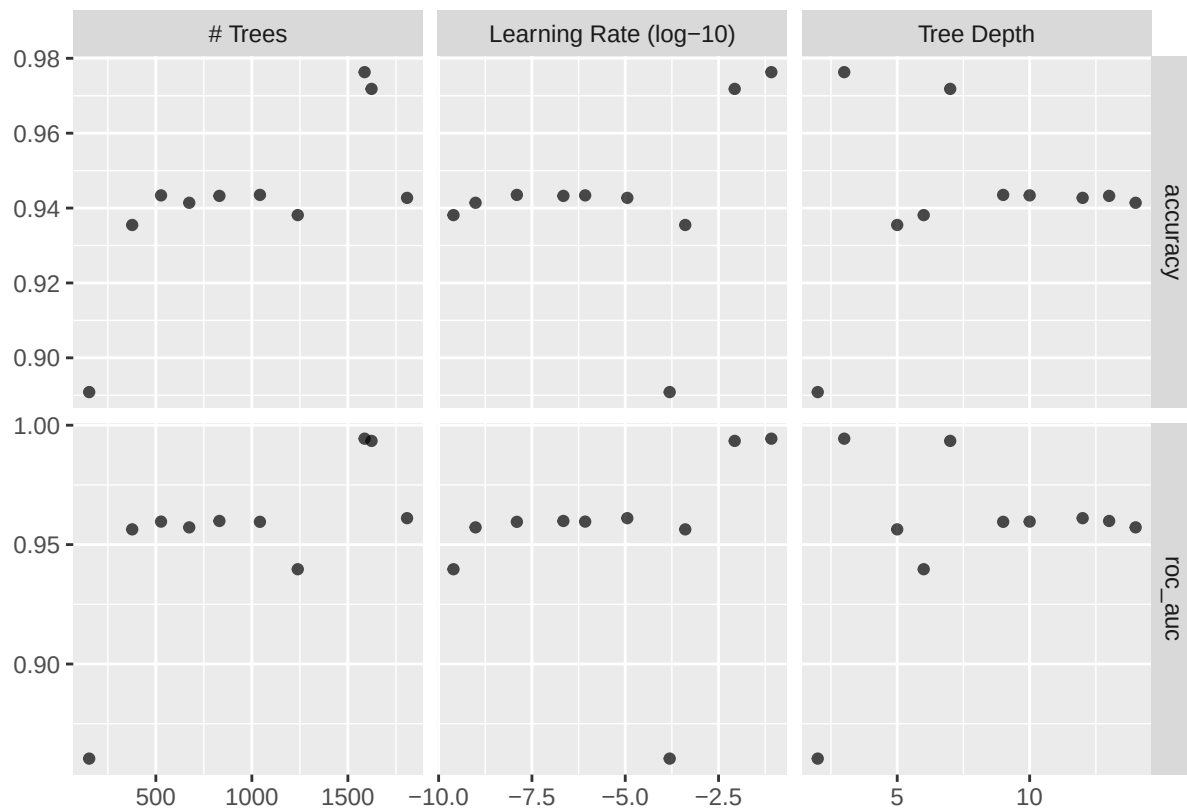


~ took 30 min to tune 10 models on professor's home computer

```
xgboost_results %>% show_best(metric = "roc_auc")
```

```
## # A tibble: 5 x 9
##   trees tree_depth learn_rate .metric .estimator  mean     n std_err .config
##   <int>    <int>      <dbl> <chr>  <chr>    <dbl> <int>  <dbl> <chr>
## 1  1587         3  0.0822  roc_auc binary  0.994    10 0.000475 Preproce~
## 2  1623         7  0.00857  roc_auc binary  0.993    10 0.000592 Preproce~
## 3  1808        12  0.0000114  roc_auc binary  0.961    10 0.00337  Preproce~
## 4   831        13  0.000000220  roc_auc binary  0.960    10 0.00360  Preproce~
## 5   527        10  0.000000844  roc_auc binary  0.960    10 0.00361  Preproce~
```

```
autoplot(xgboost_results)
```



```
best_auc <- xgboost_results %>% select_best(metric = "roc_auc")
best_acc <- xgboost_results %>% select_best(metric = "accuracy")
```

use best hyperparameter values

```
final_xgboost_workflow <- finalize_workflow(
  xgboost_workflow,
  best_auc
)
# ^ this output isn't readable, but its valid
```

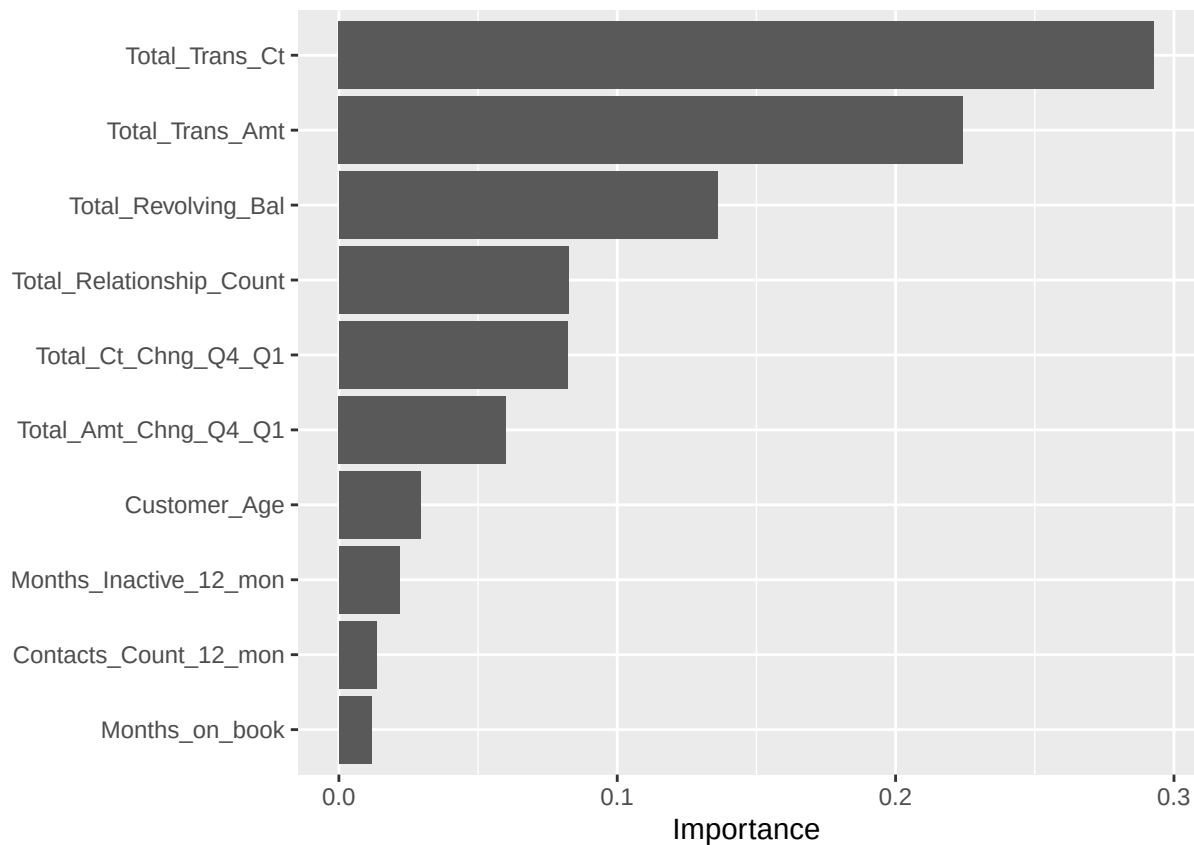
```
last_xgboost_fit <-
  final_xgboost_workflow %>%
  last_fit(data_split)
```

```
xgboost_metrics <- last_xgboost_fit %>% collect_metrics()
```

^ last_fit will train on train split and validate on test split w/o requiring specific hardcoding in

feature importance

```
last_xgboost_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 10) # check this number
```



```
# compare to the null model
null_classification <- null_model() %>%
  set_engine("parsnip") %>%
  set_mode("classification")

null_xb <- workflow() %>%
  add_recipe(attrition_recipe_xb) %>%
  add_model(null_classification) %>%
  fit_resamples(
    cv_set
  )

null_metrics <- null_xb %>% collect_metrics()
```

```
##### plot ROC curve #####
# same as other times we've plotted curve, substituting the xgboost
test_results <-
  test_set %>%
  select(Attrition_Flag) %>%
  bind_cols(
    last_xgboost_fit %>% collect_predictions() %>%
    select(p_1 = .pred_1)
  )

roc_data <- data.frame(threshold=seq(1,0,-0.01), fpr=0, tpr=0)
for (i in roc_data$threshold) {
```

```

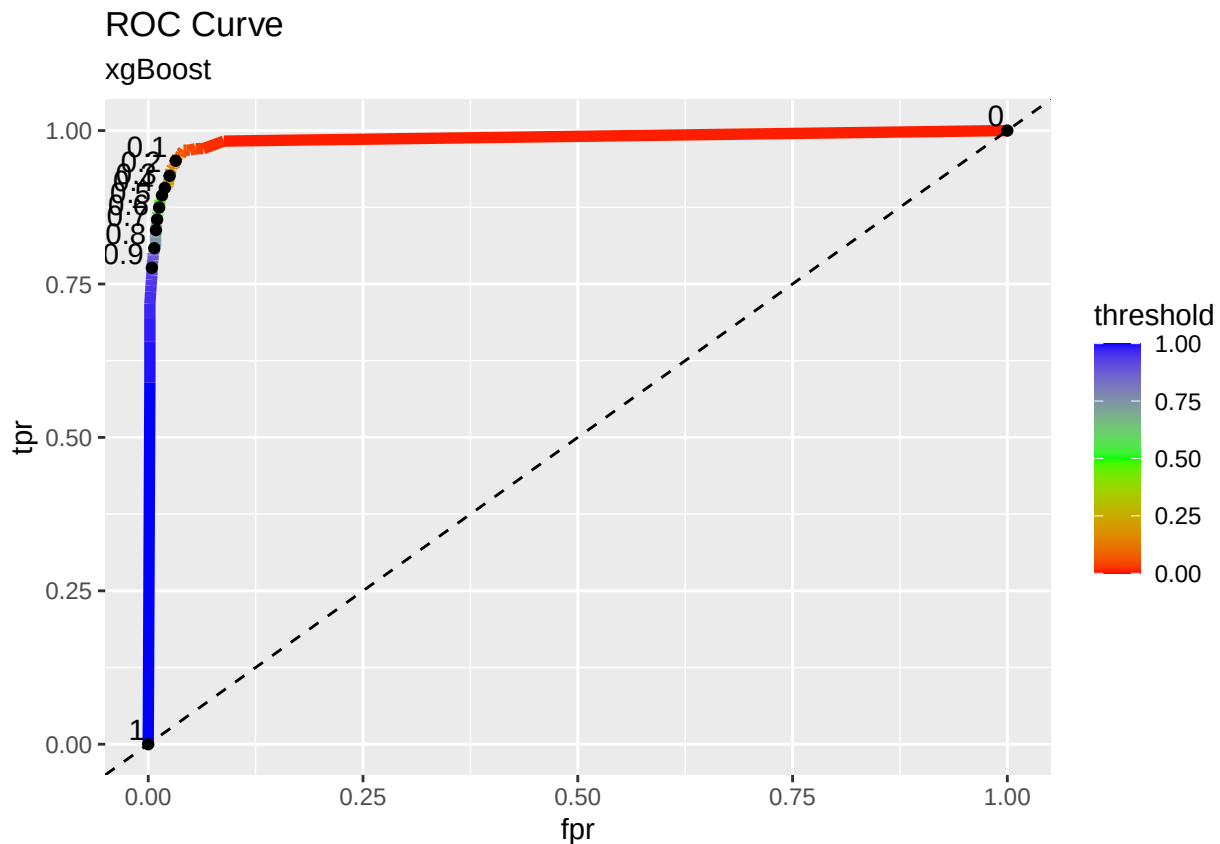
over_threshold <- test_results[test_results$p_1 >= i, ]

fpr <- sum(over_threshold$Attrition_Flag==0)/sum(test_results$Attrition_Flag==0)
roc_data[roc_data$threshold==i, "fpr"] <- fpr

tpr <- sum(over_threshold$Attrition_Flag==1)/sum(test_results$Attrition_Flag==1)
roc_data[roc_data$threshold==i, "tpr"] <- tpr
}

ggplot() +
  geom_line(data = roc_data, aes(x = fpr, y = tpr, color = threshold), linewidth = 2) +
  scale_color_gradientn(colors = rainbow(3)) +
  geom_abline(intercept = 0, slope = 1, lty = 2) +
  geom_point(data = roc_data[seq(1, 101, 10), ], aes(x = fpr, y = tpr)) +
  geom_text(data = roc_data[seq(1, 101, 10), ],
            aes(x = fpr, y = tpr, label = threshold, hjust = 1.2, vjust = -0.2)) +
  ggtitle("ROC Curve", subtitle = "xgBoost")

```

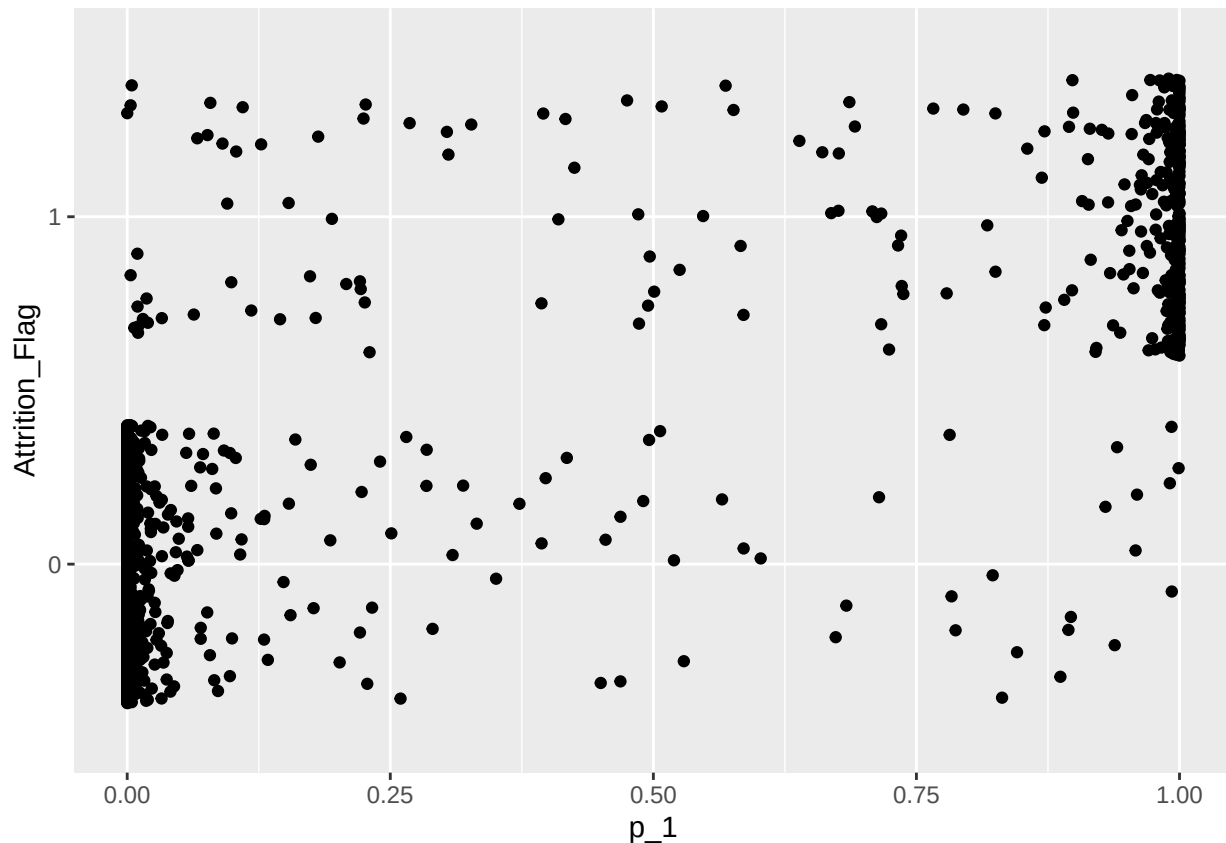


ROC curve calculation breakdown

```

ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter()

```



```
threshold <- 0.9

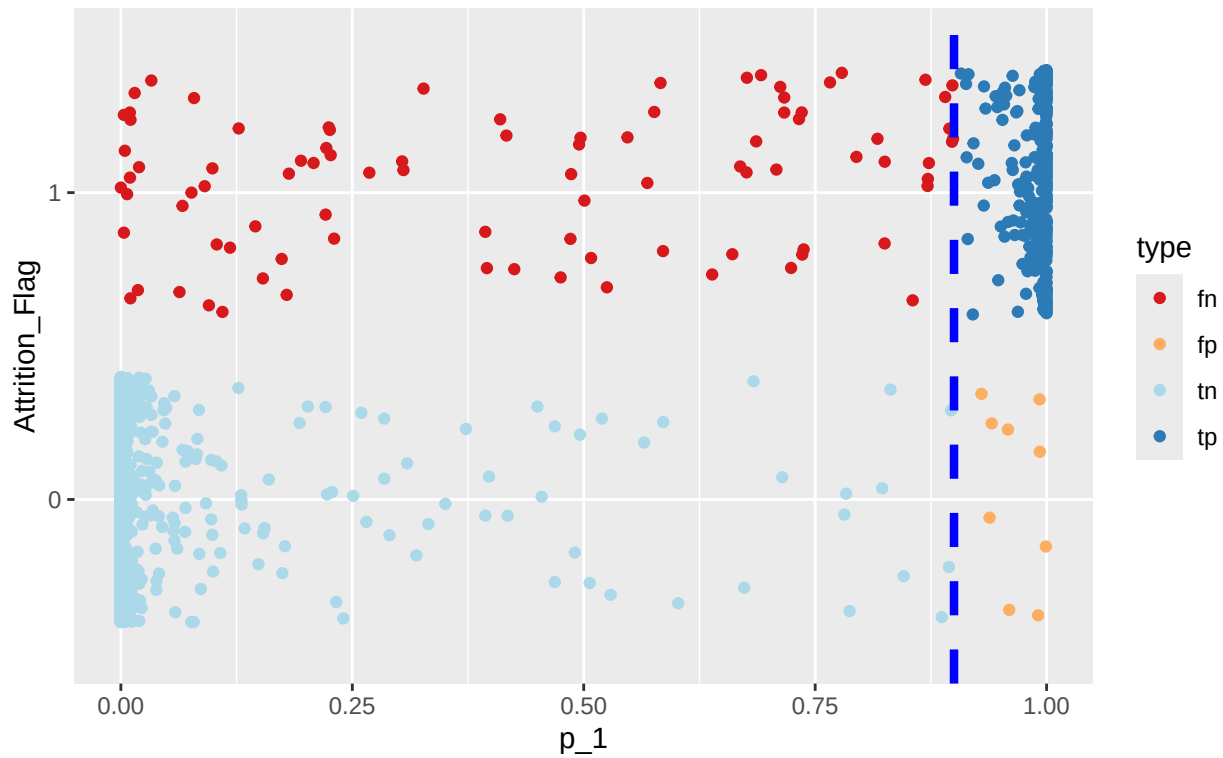
test_results$predictions <- ifelse(test_results$p_1 >= threshold, 1, 0)
tp <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, ])
fp <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, ])
tn <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, ])
fn <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, ])

test_results$type <- ""
test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, "type"] <- "tp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, "type"] <- "fp"
test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, "type"] <- "tn"
test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, "type"] <- "fn"

ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +
  geom_jitter(aes(colour = type)) +
  geom_vline(xintercept = threshold, linetype = "dashed", color = "blue", linewidth = 1.5) +
  scale_color_brewer(palette = "RdYlBu") +
  ggtitle("ROC Curve Calculation Breakdown", subtitle = "xgBoost")
```


ROC Curve Calculation Breakdown

xgBoost



```
fpr <- fp/(fp + tn)
tpr <- tp/(tp + fn)
```

```
# confusion matrix data
tn
```

```
## [1] 2116
```

```
fn
```

```
## [1] 91
```

```
fp
```

```
## [1] 9
```

```
tp
```

```
## [1] 316
```

```
# test_results_fulldata <-
#   test_set %>%
#   select (Attrition_Flag,
#           Customer_Age,
```

```

#     Gender,
#     Dependent_count,
#     Education_Level,
#     Marital_Status,
#     Income_Category,
#     Card_Category,
#     Months_on_book,
#     Total_Relationship_Count,
#     Months_Inactive_12_mon,
#     Contacts_Count_12_mon,
#     Credit_Limit,
#     Total_Revolving_Bal,
#     Avg_Open_To_Buy,
#     Total_Amt_Chng_Q4_Q1,
#     Total_Trans_Amt,
#     Total_Trans_Ct,
#     Total_Ct_Chng_Q4_Q1,
#     Avg_Utilization_Ratio) %>%
#     bind_cols(
#       last_xgboost_fit %>% collect_predictions() %>%
#       select(p_1 = .pred_1)
#     )
#
# threshold <- 0.9
#
# test_results_fulldata$predictions <- ifelse(test_results_fulldata$p_1 >= threshold, 1, 0)
# tp <- nrow(test_results_fulldata[test_results_fulldata$Attrition_Flag==1 & test_results_fulldata$predictions==1,])
# fp <- nrow(test_results_fulldata[test_results_fulldata$Attrition_Flag==0 & test_results_fulldata$predictions==1,])
# tn <- nrow(test_results_fulldata[test_results_fulldata$Attrition_Flag==0 & test_results_fulldata$predictions==0,])
# fn <- nrow(test_results_fulldata[test_results_fulldata$Attrition_Flag==1 & test_results_fulldata$predictions==0,])
#
# test_results_fulldata$type <- ""
# test_results_fulldata[test_results_fulldata$Attrition_Flag==1 & test_results_fulldata$predictions==1,]$type <- "TP"
# test_results_fulldata[test_results_fulldata$Attrition_Flag==0 & test_results_fulldata$predictions==1,]$type <- "FP"
# test_results_fulldata[test_results_fulldata$Attrition_Flag==0 & test_results_fulldata$predictions==0,]$type <- "TN"
# test_results_fulldata[test_results_fulldata$Attrition_Flag==1 & test_results_fulldata$predictions==0,]$type <- "FN"
#
# write.csv(test_results_fulldata, "~/00 Merrimack/DSE6111/Course Project/xgBoost_test_results_fulldata.csv")

```